

SLUB NOTES 1

以下所有讨论基于Linux内核v5.17

2026.4.8

SLUB allocator

SLUB分配器属于Linux内核分配器的一种。常见的分配器包括：

1. Page/Buddy allocator
2. Slab allocator
3. Vmalloc allocator

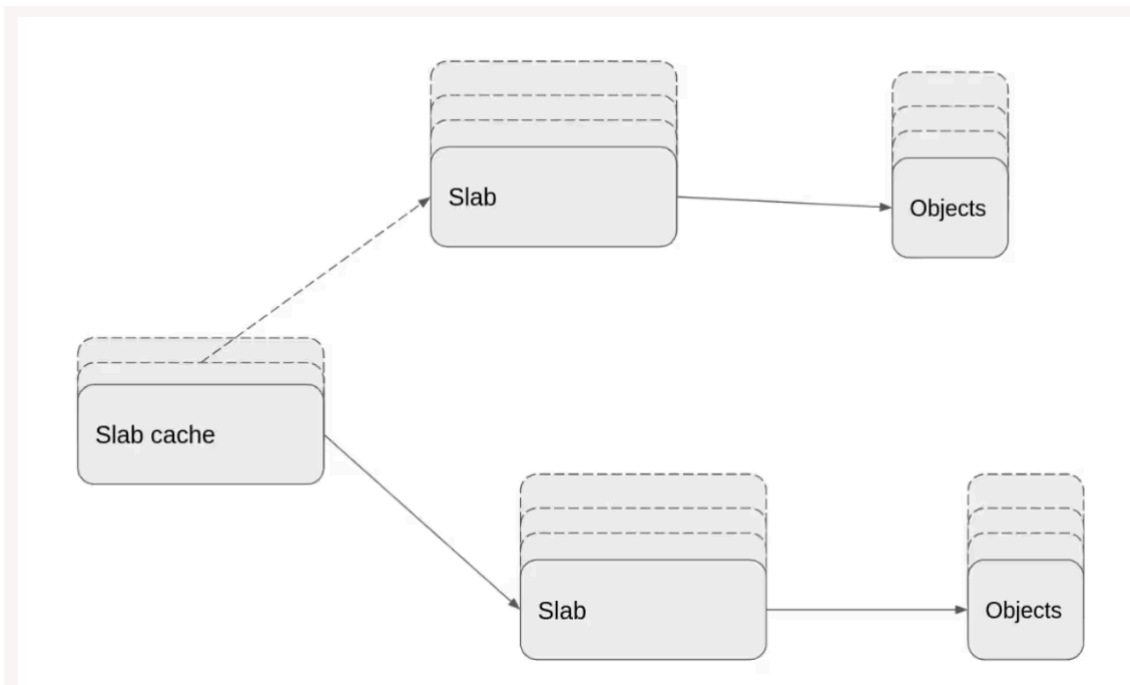
ps: slab/slub/slob用于不同情景下的内存分配，其中slub最常用。三者统称为slab分配器。

分配连续的物理页用page/buddy allocator，分配大块的不可连续的物理页用vmalloc allocator，分配较小且常用的对象用slab allocator。

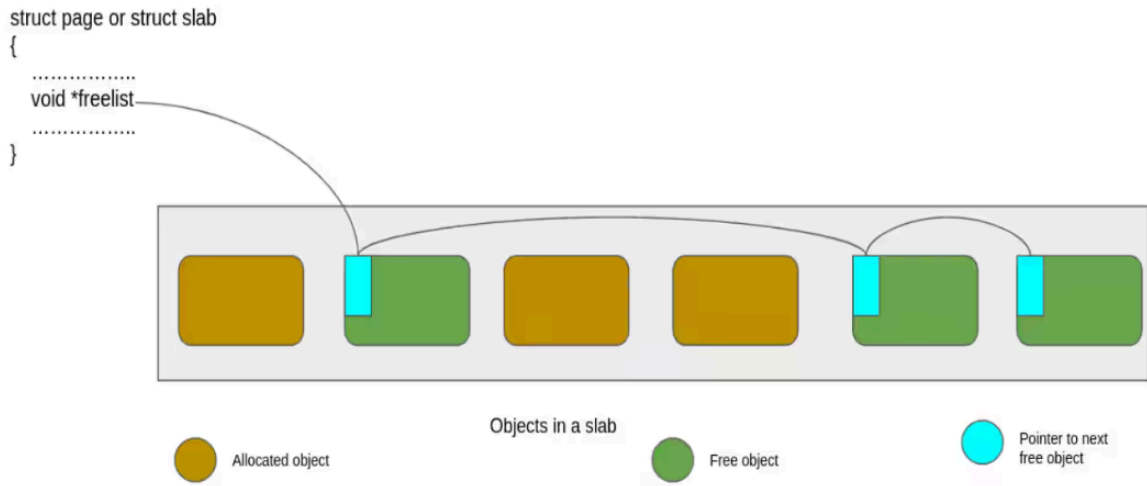
slab allocator有如下优点：

1. 避免了内核为一些常用的small object频繁分配多个page造成内存碎片和空间浪费。
2. slab allocator加速了这些常用object的分配。slab allocator使用page/buddy allocator预分配的内存页用作slab cache。

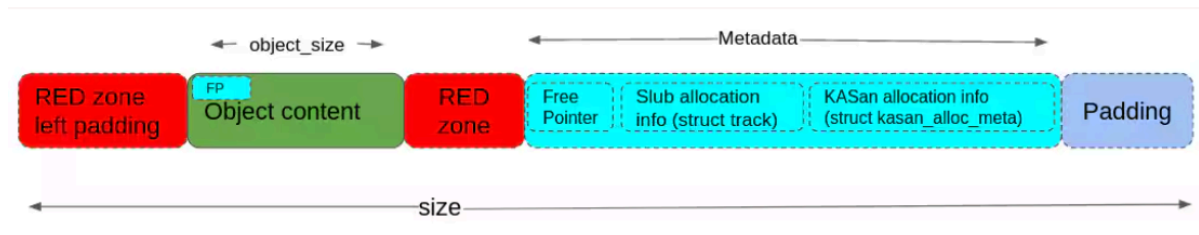
slab cache layout



每个slab cache包含一个或多个slab。每个slab又包含一个或多个page，这些page里面是切分好的固定大小的object。



slab结构体中的freelist指针指向第一个空闲的object并由object中的FP（空闲指针）指向下一个空闲指针，以此维护一个空闲链表。那么每个object的结构又是怎样的呢？



如上图，当slub_debug选项关闭时，每个object只包含Object content。当slub_debug打开时，每个object额外包含一些域用于调试，例如其中RED ZONE可以用于检测OOB漏洞。

每个slab cache对应一个kmem_cache结构体。该结构体的大致结构如下：

```

struct kmem_cache {
    struct kmem_cache_cpu __percpu *cpu_slab;
    unsigned int size;
    unsigned int object_size;
    unsigned int offset;
    struct kmem_cache_order_objects oo;
    unsigned int red_left_pad;
    // ...
    struct kmem_cache_node *node[MAX_NUMNODES];
};

```

每个kmem_cache结构体都含有一个per-CPU指针cpu_slab，该指针指向一个kmem_cache_cpu结构体，包含特定CPU对该slab cache的一些利用信息（如freelist）。而kmem_cache_node可以看成全局的内存池。这两个结构体的大致结构如下：

```

struct kmem_cache_cpu {
    void **freelist;
    struct slab *slab; /* The slab from which we are allocating */
    struct slab *partial; /* Partially allocated frozen slabs */
    local_lock_t lock;
    // ...
};

struct kmem_cache_node {
    spinlock_t list_lock;
    struct list_head slabs_partial;
};

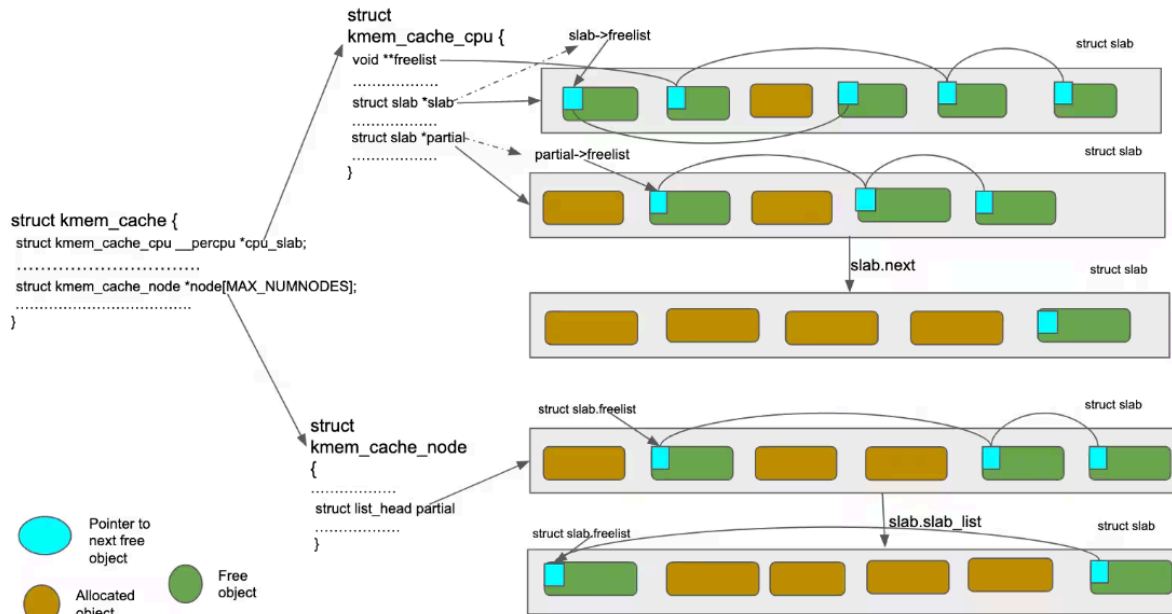
```

```

    struct list_head slabs_full;
    struct list_head slabs_free;
    // ...
};

```

ps: 由于kmem_cache_cpu中的锁仅用来防止当前CPU上的抢占，而kmem_cache_node中的锁在多个CPU的本地缓存都用完时会被抢夺而成为性能瓶颈，因此后者的开销大于前者。



一个slab有full, partially full, empty三种状态，由字面意思容易理解。其中kmem_cache_cpu->slab指向的slab又被称为per-CPU active slab。

ps: 由上图，kmem_cache_cpu->freelist和kmem_cache_cpu->slab->freelist这两个空闲链表指向active slab中的不同object。原因在后文中讲述。

Allocating a slab object

目前我们已经讨论了slab cache的布局和其中的具体结构。现在来看slub allocator是如何进行分配的:)

1. 当kmem_cache_cpu->freelist(per-CPU freelist)不为空。

该freelist中的第一个object被分配，freelist中下一个空闲object成为链表头节点。若分配后链表为空，则链表变为NULL直到下次分配。该分配无同步机制。

2. 当kmem_cache_cpu->freelist为空，但active slab的freelist不为空。

active slab的freelist中的第一个空闲object被分配，其余object被转移至per-CPU freelist，active slab的freelist成为NULL。该分配需要获得kmem_cache_cpu.lock，因此较1而言略慢。

3. 当active slab中没有空闲object，但per-CPU partial slab list存在有空闲object的slab。

per-CPU partial slab list中第一个存在空闲object的slab成为active slab，其freelist成为per-CPU freelist，该freelist中的第一个object被分配。该分配的速度较2略慢。原来的active slab变为full slab，不被任何链表追踪，因为我们不关心一个full slab。

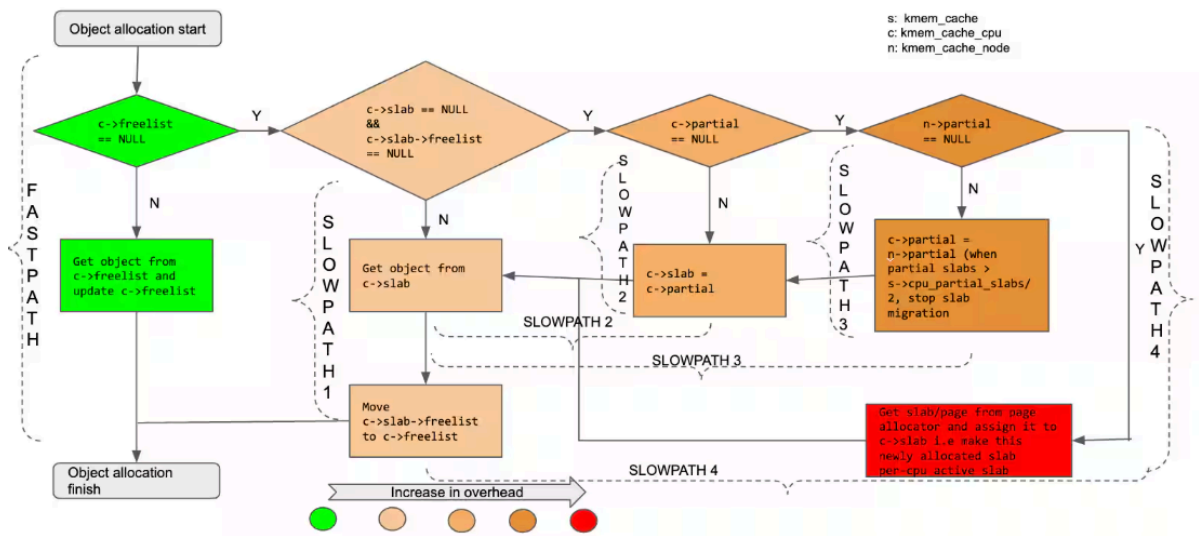
4. 当per-CPU slabs(active and partial)都没有空闲object。

slub allocator尝试从全局内存节点 (kmem_cache_node) 寻找一个离该CPU最近的含有partial slab的节点，对第一个找到的partial slab，将第一个free object用作被分配的对象，该slab成为per-CPU active slab。该分配的遍历全局kmem_cache_node过程需要获得kmem_cache_node.list_lock，因此较2, 3慢了很多。此外，在分配结束后，slub allocator会不断

地将该node的全部partial slab转移至per-CPU的partial slab list，直到全部转移完或per-CPU partial slab list中的partial slab数量达到上限。

5. 当整个slab cache都满了。

page allocator会分配一个新的slab，这个新的slab成为CPU的active slab。该分配过程最慢，因为需要buddy allocator分配新的页。



Freeing a slab object

与分配类似，释放一个object也存在如下情形：

- per-cpu active slab
- per-cpu partial slab list
- per-node partial slab list
- full slab

被释放的object会被插到相应的freelist的头部。

1. object在active slab中，并且释放该object的CPU正是active slab对应的CPU

该object进入`kmem_cache_cpu->freelist`。

2. object在active slab中，但释放该object的CPU不是active slab对应的CPU

该object进入`kmem_cache_cpu->slab->freelist`。

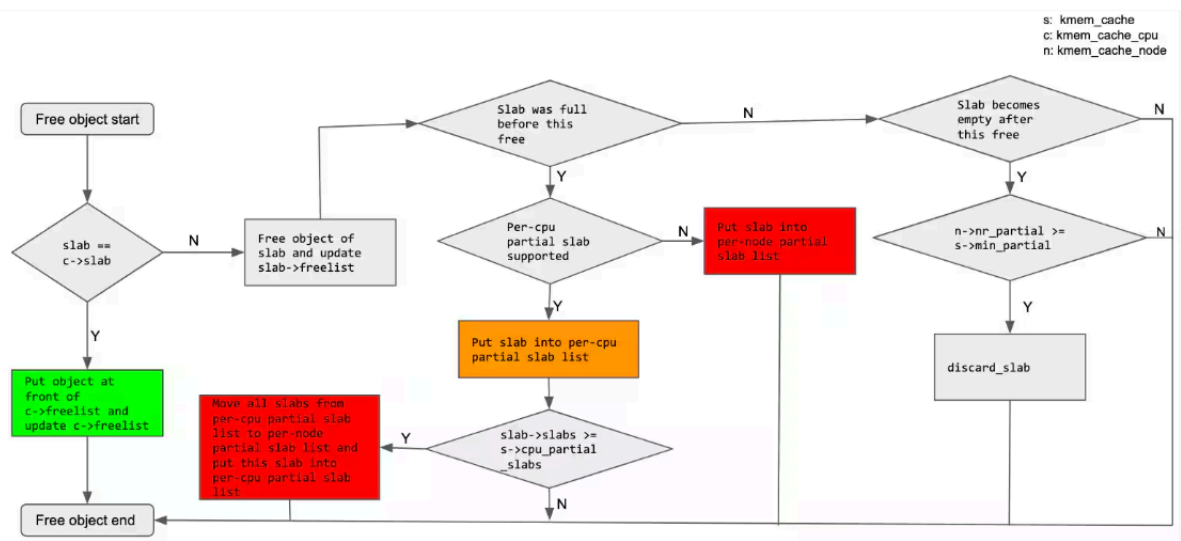
3. object在一个full slab中

该slab成为一个partial slab，并加入per-CPU partial slab list或者per-node partial slab list（视情况而定）。若为per-CPU partial slab list且该list已满，则需要将该list中原来的所有partial slab移至全局节点（per-node partial slab list），然后再将此新的partial slab加入per-CPU partial slab list。

4. object在一个partial slab中

- 释放后仍然是partial
将object加入相应的freelist即可。
- 释放后变为empty

若该empty slab所处的partial slab list的slab总数小于slab cache's `min_partial`，那么该empty slab仍保留在该partial slab list中，否则被释放并从该partial slab list中移除。



References

- [1] <https://blogs.oracle.com/linux/linux-slab-allocator-internals-and-debugging-1>
- [2] <https://elixir.bootlin.com/linux/v5.17/source>